EMPLOYEE ATTENDANCE DASHBOARD

Technology Notes

What technology is used, why we used it, and how it all fits together

Project: Employee Attendance Dashboard (AttendTrack)

Made by: Cecil Baraik, B.Sc. IT

Stack: Java + Spring Boot + React + MySQL + Docker

Contents

1. Project in one page 3
2. Architecture (how the parts talk) 4
3. Backend technologies 6
4. Frontend technologies 10
5. Deployment technologies 11
6. Important concepts used 13
7. Security features (summary) 14
8. Folder structure 16
9. Attendance rules (business logic) 16
10. Limitations and future scope 17

1. Project in one page

Project name: Employee Attendance Dashboard (app name: **AttendTrack**)

Problem: Many offices still use a paper register or an Excel sheet for attendance. It is slow, easy to fake (a friend can sign for you), hard to calculate monthly reports, and there is no record of who changed what.

Solution: A web application where employees check in and out, managers approve leave and corrections, and admins see live dashboards and download reports. It also stops fake check-ins.

Main features

#	Feature	Short explanation
1	Security and access	Login with JWT, 3 roles (Admin, Manager, Employee), BCrypt passwords, account lock, audit log.
2	Employee self-service	Self check-in / check-out, leave requests with balance, correction requests, manager approvals.
3	Fake check-in prevention	Rotating QR code (kiosk), GPS geofence, office network (IP) check.
4	Smart rules	Holidays, shifts (including night shift), late / half day / overtime calculation.
5	Reports and alerts	Month calendar, CSV / Excel / PDF export, "needs attention" list, email alerts on a schedule.
6	Production setup	MySQL, Docker, nginx, health check, environment variables, deployment guide.
+	Light / dark mode	Theme switch that remembers your choice.

Technology summary

Layer	Technology
Frontend (what the user sees)	React 18, Vite 5, React Router, Recharts, qrcode library, CSS variables
Backend (server logic)	Java 17+, Spring Boot 3.3, Spring Web, Spring Security, JWT (jjwt), Spring Data JPA / Hibernate, Bean Validation, Spring Mail, Scheduling, Apache POI, OpenPDF, Actuator
Database	H2 (development), MySQL 8 (production)
Build tools	Maven (backend), npm (frontend)
Testing	JUnit 5, Spring Boot Test, MockMvc, AssertJ - 47 automated tests
Deployment	Docker, Docker Compose, nginx
Version control	Git and GitHub

2. Architecture (how the parts talk)

The project uses a **client-server** architecture with a **3-tier** design:

```
+-----------------------------------------+
|  FRONTEND - React (runs in the browser)   |   http://localhost:5173
+-----------------------------------------+
           |   HTTP request + JSON
           |   Authorization: Bearer <JWT token>
           v
+-----------------------------------------+
|  BACKEND - Spring Boot REST API           |   http://localhost:8080
|  Controller  ->  Service  ->  Repository  |
+-----------------------------------------+
           |   SQL (generated by JPA / Hibernate)
           v
+-----------------------------------------+
|  DATABASE - H2 (development) / MySQL      |
+-----------------------------------------+
```

Tier	Job	In this project
Presentation tier	Shows screens, takes clicks and input	React frontend
Business / logic tier	Rules, security, calculations	Spring Boot backend
Data tier	Stores data permanently	H2 or MySQL database

Layers inside the backend

The backend follows the **layered architecture** (a common Spring pattern, similar to MVC):

Layer (package)	Job	Example
controller	Receives the HTTP request, sends back JSON. No business logic.	AttendanceController, AuthController
service	Business rules: is this check-in late? Is leave balance enough?	AttendanceService, LeaveService, WorkCalendar
repository	Talks to the database (save, find, delete).	EmployeeRepository, AttendanceRecordRepository
model	Java classes that map to database tables (entities).	Employee, AttendanceRecord, LeaveRequest
dto	Small objects for request/response data.	Login request, attendance view
security	Login, JWT, permissions.	SecurityConfig, JwtService, AccessGuard
config	Settings, demo data, CORS, scheduling.	DataSeeder, WebConfig
exception	Turns errors into clean JSON messages.	GlobalExceptionHandler

> **Remember:** Why layers? Each layer has one job, so code is easy to read, test and change. For example, we moved from H2 to MySQL without touching the controller or service code.

Example: what happens when an employee clicks "Check in"

1. React sends `POST /api/me/check-in` with the JWT token in the `Authorization` header (and the QR code / GPS if needed).
2. **JwtAuthFilter** checks the token signature and expiry, and finds the logged-in user.
3. Spring Security checks the role is allowed (`@PreAuthorize`).
4. **CheckInPolicyService** checks the security rules: QR code correct? GPS inside the office radius? IP inside the allowed range? If not, it rejects and writes "Check-in rejected" in the audit log.
5. **AttendanceService** finds the employee's shift (**WorkCalendar**), decides Present or Late, and saves an **AttendanceRecord** through the repository.
6. Hibernate turns this into an SQL `INSERT` in the database.
7. The backend returns JSON; React updates the screen and shows a message.

Database tables (entities)

Table / Entity	What it stores
Employee	Code, name, email, department, designation, shift, join date, active
AttendanceRecord	Employee, date, check-in, check-out, status, note, how it was verified
UserAccount	Username, BCrypt password hash, role, linked employee, failed attempts, lock time, token version
LeaveRequest	Employee, type, from/to dates, days, reason, status, reviewer note
CorrectionRequest	Employee, date, requested times, reason, status
Holiday	Date and name
Shift	Name, start, end, grace minutes, standard hours
CheckInSettings	QR / GPS / network rules and their values
Notification	Email outbox: to, subject, body, status (Sent / Logged / Failed)
AuditLog	Who, when, action, details, IP address

Relationships: one **Employee** has many **AttendanceRecords** and many **LeaveRequests** (one-to-many). One **UserAccount** is linked to one Employee (one-to-one). Many employees can have the same **Shift** (many-to-one).

3. Backend technologies

Java (JDK 17+)	
What is it?	An object-oriented programming language. Code runs on the JVM (Java Virtual Machine), so it works on Windows, Linux and Mac.
Why we used it	Strongly typed (fewer bugs), very fast, huge library support, used by most big companies for backend systems. Java 17 is an LTS (Long Term Support) version. We used *records* (short data classes) from modern Java.
Where in this project	All backend code: package `com.cecil.attendance`.
Other options	Python (Django), Node.js (Express), C# (.NET)

Spring Boot 3.3	
What is it?	A framework built on the Spring framework that makes Java web apps quick to build. It has an embedded Tomcat server and "auto-configuration".
Why we used it	No need to set up a server by hand; one command (`mvn spring-boot:run`) starts everything. Dependency Injection keeps classes loosely coupled. "Starters" add features with one dependency. Industry standard for Java REST APIs.
Where in this project	The whole backend. Main class: `AttendanceApplication`.
Other options	Plain Spring, Jakarta EE, Micronaut, Quarkus

Spring Web (REST API)	
What is it?	Part of Spring for building REST APIs. Uses annotations like `@RestController`, `@GetMapping`, `@PostMapping`.
Why we used it	The React frontend and the backend are separate, so they talk with HTTP and JSON. REST is simple and any client (web, mobile app) can use the same API.
Where in this project	12 controllers, for example `/api/attendance`, `/api/employees`, `/api/reports`, `/api/auth/login`.
Other options	GraphQL, gRPC

Spring Data JPA + Hibernate	
What is it?	**JPA** is the Java standard for ORM (Object Relational Mapping). **Hibernate** is the implementation. **Spring Data** creates repository code for us.
Why we used it	We work with Java objects, not raw SQL. A method name like `findByEmployeeCodeIgnoreCase` becomes a query automatically. Tables are created from the entity classes. Same code works on H2 and MySQL. Parameterised queries also protect against SQL injection.
Where in this project	Package `model` (@Entity classes) and `repository` (interfaces).
Other options	JDBC with plain SQL, MyBatis, jOOQ

Spring Security	
What is it?	The security framework for Spring apps: authentication (who are you?) and authorization (what can you do?).
Why we used it	Tested and trusted, handles filters, password encoders and role checks. We use it in **stateless** mode with JWT. `@PreAuthorize("hasRole('ADMIN')")` protects each API.
Where in this project	`SecurityConfig`, `JwtAuthFilter`, `AccessGuard` (department checks, no self-approval).
Other options	Writing security by hand (risky), Apache Shiro, Keycloak

JWT (JSON Web Token) - jjwt library	
What is it?	A signed token with 3 parts: header.payload.signature. The payload has the username, role and expiry.
Why we used it	The server does not need to store sessions (stateless), so it scales easily. The signature (HMAC-SHA256) means nobody can change the token. Works well with a separate React frontend.
Where in this project	`JwtService` creates and checks tokens; valid for 8 hours; the frontend sends it as `Authorization: Bearer <token>`.
Other options	Session cookies, OAuth2 / OpenID Connect

BCrypt	
What is it?	A slow, salted password-hashing algorithm.
Why we used it	Passwords are never stored as plain text. BCrypt adds a random salt, so two equal passwords give different hashes, and it is deliberately slow, so guessing passwords takes a very long time.
Where in this project	`BCryptPasswordEncoder` in SecurityConfig, used for every login and password change.
Other options	Argon2, PBKDF2, scrypt

Bean Validation (Jakarta Validation)	
What is it?	Annotations like `@NotBlank`, `@Email`, `@Size` on input fields.
Why we used it	Rejects bad input at the door with a clear 400 error, before it reaches business logic.
Where in this project	Request DTOs (employee form, login, leave request).

Spring Mail + Scheduling	
What is it?	**Spring Mail** sends email over SMTP. **@Scheduled** runs a method at fixed times using a cron expression.
Why we used it	Automatic reminders without anyone pressing a button. Example cron: `0 30 10 * * MON-FRI` = 10:30 on weekdays.
Where in this project	`AlertService`, `MailService`, Notification outbox table.

Apache POI and OpenPDF	
What is it?	**Apache POI** creates Microsoft Excel (.xlsx) files. **OpenPDF** creates PDF files.
Why we used it	HR people want reports in Excel and PDF, not only on screen. Both are free and open source.
Where in this project	`ExportService`: Reports > Export > Excel / PDF.

H2 Database	
What is it?	A small database written in Java. It saves to a file in `backend/data`.
Why we used it	No installation needed for development. Start the app and it works. Has a web console to see tables.
Where in this project	Default (development) profile.
Other options	SQLite, an installed MySQL

MySQL 8	
What is it?	A popular open-source relational database server.
Why we used it	Reliable for many users at once, free, supported by every hosting company. Used in production.
Where in this project	`prod` profile (`application-prod.properties`) and Docker.
Other options	PostgreSQL, Oracle, SQL Server

Spring Boot Actuator	
What is it?	Adds monitoring endpoints to the app.
Why we used it	Docker and servers need to know "is the app healthy?". Only `/actuator/health` is public; everything else is blocked for security.
Where in this project	Docker HEALTHCHECK, SecurityConfig.

Maven	
What is it?	Build and dependency tool for Java. The file `pom.xml` lists the libraries.
Why we used it	Downloads all libraries automatically, compiles, runs tests (`mvn test`) and builds the jar.
Where in this project	`backend/pom.xml`.
Other options	Gradle

JUnit 5 + Spring Boot Test + MockMvc	
What is it?	Testing tools. JUnit runs tests, Spring Boot Test starts the app for tests, MockMvc sends fake HTTP requests.
Why we used it	Proves the rules work and stay working after changes. 47 tests cover attendance rules, security, leave, check-in policy, shifts, reports and production settings.
Where in this project	`backend/src/test/java`.

4. Frontend technologies

React 18	
What is it?	A JavaScript library (by Meta) for building user interfaces from reusable **components**.
Why we used it	The screen updates automatically when data changes (state). Components like StatusBadge, Modal and CalendarGrid are written once and reused on many pages. Very popular, so lots of help and jobs.
Where in this project	`frontend/src`: 15 pages and 9 shared components. Hooks used: `useState`, `useEffect`, `useContext`.
Other options	Angular, Vue.js, plain HTML + JavaScript

Vite 5	
What is it?	A build tool and development server for modern JavaScript.
Why we used it	Starts in under a second and reloads the page instantly when you save a file. Builds small, optimised files for production. Its **proxy** forwards `/api` calls to port 8080 in development.
Where in this project	`vite.config.js`, `npm run dev`, `npm run build`.
Other options	Create React App (old), Webpack

React Router	
What is it?	Library for moving between pages in a Single Page Application (SPA) without reloading.
Why we used it	Each page has its own URL (`/reports`, `/my`). We show only the routes allowed for the user's role.
Where in this project	`App.jsx`.

Recharts	
What is it?	A chart library built for React.
Why we used it	Easy to make bar charts and graphs that change colour with the light/dark theme.
Where in this project	Dashboard: daily attendance chart, department rates.

qrcode library	
What is it?	Generates a QR code image in the browser.
Why we used it	The kiosk screen shows a new QR code every 30 seconds.
Where in this project	`Kiosk.jsx`.

CSS variables (light / dark mode)	
What is it?	Colours are stored as variables like `--bg`, `--text`.
Why we used it	To switch theme we only change one attribute on the page: `<html data-theme="dark">`. All colours change at once. The choice is saved in localStorage; the first time it follows the computer's setting (`prefers-color-scheme`).
Where in this project	`index.css`, `theme.jsx`.

Fetch API + localStorage	
What is it?	**fetch** sends HTTP requests from the browser. **localStorage** keeps small data in the browser.
Why we used it	All API calls go through one file (`api.js`) that adds the token and handles errors. If the server says 401, the user is sent back to login.
Where in this project	`api.js`, `auth.jsx`.

5. Deployment technologies

Docker	
What is it?	Packs an application with everything it needs (Java, libraries, settings) into an **image**. A running image is a **container**.
Why we used it	"It works on my computer" problem is solved: the same image runs the same way everywhere. We used **multi-stage builds** (build with Maven/Node, then copy only the result into a small image) and run as a non-root user.
Where in this project	`backend/Dockerfile`, `frontend/Dockerfile`.
Other options	Installing everything on the server by hand, virtual machines

Docker Compose	
What is it?	Starts many containers together from one file.
Why we used it	One command (`docker compose up -d --build`) starts MySQL, backend and frontend in the right order (waits until the database is healthy). Database and backend are not open to the internet.
Where in this project	`docker-compose.yml` and the `.env` file.
Other options	Kubernetes (for big systems)

nginx	
What is it?	A fast web server and reverse proxy.
Why we used it	Serves the built React files, forwards `/api` to the backend, and adds security headers (Content-Security-Policy, X-Frame-Options...). Blocks actuator and H2 console from outside.
Where in this project	`frontend/nginx.conf`.
Other options	Apache HTTP Server, Caddy

Git and GitHub	
What is it?	Git tracks changes in code (version control). GitHub stores the repository online.
Why we used it	History of every change, backup, and easy to share with teachers and recruiters. We committed after each feature.
Where in this project	Repo: `cecilbaraik19/Web-Development`, folder `Java Projects/attendance-dashboard`.

6. Important concepts used

REST API

REST = Representational State Transfer. Data is reached by URLs and HTTP methods:

Method	Meaning	Example in this project
GET	Read data	`GET /api/employees` - list employees
POST	Create / do an action	`POST /api/auth/login`, `POST /api/me/check-in`
PUT	Update	`PUT /api/employees/5` - edit employee 5
DELETE	Remove	`DELETE /api/employees/5`

HTTP status codes used: **200** OK, **201** Created, **400** Bad input, **401** Not logged in, **403** No permission, **404** Not found, **409** Conflict (e.g. already checked in), **429** Too many tries.

Authentication vs Authorization

- **Authentication** = proving who you are (username + password, then JWT).
- **Authorization** = what you are allowed to do (Admin vs Manager vs Employee, own department only).

Role-Based Access Control (RBAC)

Permissions are given to roles, and users get a role. The check happens in two places: the frontend hides menu items, and the **backend** really blocks the API (the frontend alone is never trusted).

Rotating QR code (TOTP idea)

Same idea as Google Authenticator. The server has a secret key. Every 30 seconds it calculates:

```
code = HMAC-SHA256(secret, current_time / 30)  ->  6 digits
```

Without the secret, nobody can predict the next code. The server accepts the current and the previous code (for slow scans), compares in constant time (stops timing attacks), and blocks after 5 wrong tries for 10 minutes.

Geofence (Haversine formula)

The Haversine formula finds the distance between two GPS points on the Earth (a sphere). If the distance between the phone and the office is more than the radius (e.g. 200 m), check-in is refused.

CIDR (IP ranges)

`192.168.1.0/24` means all addresses from 192.168.1.0 to 192.168.1.255. The first 24 bits must match. Used for the office network rule.

CORS

Cross-Origin Resource Sharing. A browser blocks a page from calling an API on a different address unless the API allows it. We allow only our frontend address.

Stateless backend

The server keeps no login session in memory. Every request carries the JWT. This makes it easy to restart the server or run more copies.

Dependency Injection (DI)

Spring creates objects (beans) and passes them into constructors. Example: `AttendanceService` receives the repository; it never writes `new`. This makes testing and changes easy.

Profiles and environment variables

dev profile = H2, demo data. **prod** profile = MySQL, secrets from environment variables, no demo passwords, no stack traces in errors. Passwords are never written in the code.

Audit log

Every important action is saved with user, time and IP. This gives **accountability** - you can always find who changed a record.

7. Security features (summary)

Threat	Protection in this project
Stolen password database	BCrypt hashes, not plain passwords
Password guessing (brute force)	Account lock after 5 wrong tries for 15 min; QR code lock after 5 wrong codes
Weak passwords	Password policy: 8+ characters, upper, lower, number, not the username
Changing a token	JWT is signed with HMAC-SHA256; secret must be 32+ characters in production
Old token after password change	Token version: changing or resetting a password logs out old sessions
Seeing other departments	AccessGuard checks the manager's department on every request
Approving own leave	Blocked in AccessGuard
Buddy punching	Rotating QR, GPS geofence, office IP rule

Threat	Protection in this project
Fake IP header (X-Forwarded-For)	Trusted only from known proxies
SQL injection	JPA parameterised queries
XSS / clickjacking	React escapes output; nginx CSP and X-Frame-Options headers; plain-text emails
Leaking server details	No stack traces in prod; only /actuator/health public; H2 console off in prod
Hidden changes	Audit log of every action

8. Folder structure

```
attendance-dashboard/
|-- backend/                    Spring Boot API
|   |-- pom.xml                 Maven: list of libraries
|   |-- Dockerfile
|   `-- src/
|       |-- main/java/com/cecil/attendance/
|       |   |-- AttendanceApplication.java   (main method)
|       |   |-- model/          Entities (tables)
|       |   |-- repository/     Database access
|       |   |-- service/        Business logic
|       |   |-- controller/     REST APIs
|       |   |-- dto/            Request / response objects
|       |   |-- security/       JWT, Spring Security, AccessGuard
|       |   |-- config/         Settings, demo data, scheduling
|       |   `-- exception/      Error handling
|       |-- main/resources/
|       |   |-- application.properties        (development)
|       |   `-- application-prod.properties   (production)
|       `-- test/java/          47 automated tests
|-- frontend/                   React app
|   |-- package.json            npm: list of libraries
|   |-- vite.config.js
|   |-- nginx.conf, Dockerfile
|   `-- src/
|       |-- App.jsx             Routes + sidebar
|       |-- api.js              All calls to the backend
|       |-- auth.jsx            Login state
|       |-- theme.jsx           Light / dark mode
|       |-- index.css           Styles and colour variables
|       |-- pages/              Dashboard, Attendance, Reports, Kiosk ...
|       `-- components/         Modal, Toast, StatusBadge, CalendarGrid ...
|-- docker-compose.yml          MySQL + backend + frontend
|-- .env.example                Settings template for Docker
|-- start-dev.ps1               Start both parts on Windows
|-- README.md, DEPLOYMENT.md
`-- docs/screenshots/
```

9. Attendance rules (business logic)

Rule	Value / formula
Office start (no shift)	09:30
Grace time	15 minutes, so Late after 09:45
Half day	Worked less than 4.5 hours
Standard hours	8 hours (or the shift's own value)
Hours worked	check-out - check-in (night shift: adds 24 h if check-out is after midnight)
Overtime	hours worked - standard hours; all hours on weekend / holiday

Rule	Value / formula
Weekend	Saturday and Sunday
Leave per year	Casual 12, Sick 10, Earned 15, Unpaid unlimited
Leave days	Counts working days only (no weekends or holidays)
Attendance %	days attended (present + late + half day) / (working days - leave days) x 100
Needs attention	Late or absent 3 or more times in the period

10. Limitations and future scope

Limitations

- No fingerprint or face recognition, so a shared password plus physical presence can still cheat.
- No payroll / salary calculation.
- No native mobile app (the website works on phones).
- Single company (not multi-company / SaaS).
- GPS can be faked on rooted phones (that is why it is combined with QR and IP rules).

Future scope

- Face recognition or biometric device integration.
- Payroll module using hours, overtime and leave.
- Android / iOS app with push notifications.
- Two-factor authentication (2FA) for admins.
- Multi-company support and cloud hosting (AWS / Azure).
- CI/CD pipeline with GitHub Actions to run tests on every push.